

Meteor dodge game debugging

10/07/2026 14:05:29

<https://claude.ai/chat/ddbcd5a3-85d9-43f9-81da-3cf2ffefc5d>

<https://github.com/SETAPESU26/38-meteor-dodge> HEYY HELP ME DOING...

I'll read the repo's README first so the steps match what your game...

GENERATE THE CODE TO PASTE IT AND SOLVE THE BUG

I'll pull the actual source files so the fix matches your code instead of...



<https://github.com/SETAPESU26/38-meteor-dodge> HEYY HELP ME DOING THIS SHLD I DO IT ON MY OWN OR ELSE AUTOMATE IT TO ANTOGRAVITY ? HELP ME DO EVERTHING GIVE ALL THE BASIC STEPS , FIRST WE SHLD PLAY GAME RECORD FOR 10 SEC LATER FIND THE BUGS SOLVE IT RECORD FOR 10 SEC

10/7/2026, 11:50:47 AM



I'll read the repo's README first so the steps match what your game actually needs.

Weighing manual tracking against an automated tool for the workflow. ▾

🔗 Read a page

🕒 Weighing manual tracking against an automated tool for the workflow.

🟢 Done

I read the repo. It's a Pygame game with one deliberate bug (Task 1: SPACE doesn't fire lasers) and three features to add (meteor splitting, shield orbs, survival multiplier).

Do it yourself or automate with Antigravity?

Do the process yourself, and use an LLM for the code. The lab grades your prompting (fix things in 3-4 attempts), separate commits, and an exported chat history. If Antigravity's agent does everything in one go, you'll have a hard time showing task-by-task prompts and commits, and the chat export is clumsy. A plain chat (Claude, Gemini or ChatGPT) gives you a clean history to export. You can still use Antigravity or VS Code as your editor and terminal.

Steps

1. Get the code

- Easiest: click **Fork** on the repo page. This puts a copy in your own account, and you never open a PR to SETAPESU26.
- Then:

```
git clone https://github.com/Pradhyut21/38-meteor-dodge.git
cd 38-meteor-dodge
pip install -r requirements.txt
python main.py
```

2. Record the "before" video (10 sec)

- Use Win+G (Xbox Game Bar), OBS, or Antigravity's screen recorder.
- Show the title screen, press SPACE to launch, move around, and press SPACE again to show that no laser fires. Let a meteor hit you and show Game Over.
- Save it as `before.mp4`.

3. Start one chat and keep it all in there

Open a fresh chat with your LLM. Paste `main.py`, `game/game_engine.py`, `game/ship.py` and `game/meteor.py` once at the start, and use the same chat for all four tasks so the export is complete.

4. Do the tasks one at a time, with one commit each

Prompt templates, adjust after you see the code:

- **Task 1 (bug):** "Here is my Pygame code (pasted). Pressing SPACE launches from the title screen but does nothing during flight. It should fire a laser projectile that destroys meteors on hit. Find the input-state bug, add a Laser class with a cooldown, and give me only the changed code with file names."
- **Task 2 (splitting):** "When a large meteor is hit by a laser, it should split into 2-3 smaller fragments that diverge outward. Small meteors should just disappear. Add a size or generation attribute to Meteor."
- **Task 3 (shield):** "Add a drifting energy orb that spawns occasionally. Collecting it gives the ship a shield that absorbs one meteor collision, then breaks. Draw a visible shield ring and show shield status in the HUD."
- **Task 4 (multiplier):** "Score should gain faster for every 10 seconds survived without being hit: x1, x2, x3, and so on. Reset the multiplier when the ship takes a hit (including a shield hit). Show the multiplier in the HUD."

After each task, run the game, check it works, and commit:

```
git add .  
git commit -m "Task 1: fix laser firing input bug"
```

Do the same with "Task 2: meteor splitting", "Task 3: shield power-up", and "Task 4: survival multiplier".

If a result is wrong, tell the LLM exactly what you saw, for example "lasers fire but pass through meteors". Staying within 3-4 attempts per task depends on specific feedback like that.

5. Record the "after" video (10 sec)

Show the laser firing, a meteor splitting, picking up the shield and surviving a hit, and the multiplier going up in the HUD. Save it as `after.mp4`.

6. Export the chat

Copy the whole conversation into a doc or PDF, or use the share link if your tool has one. The README asks for the chat link.

7. Push

Make a `Lab-4/` folder in your repo containing `before.mp4`, `after.mp4`, the chat PDF and the updated code. If your instructor wants a separate Lab repo, copy it there.

```
git add .  
git commit -m "Add Lab-4 videos and chat history"  
git push origin main
```

Don't forget the PDF and videos, since the submission is those plus the chat link.

Next step

Once you've cloned it, paste `game_engine.py` and `ship.py` here and I'll help you with Task 1. Your chat with me can be the exported history, as long as you keep it all in this one conversation.

10/7/2026, 11:51:06 AM



GENERATE THE CODE TO PASTE IT AND SOLVE THE BUG

10/7/2026, 2:02:50 PM



I'll pull the actual source files so the fix matches your code instead of guessing.

Wiring laser-meteor collision detection into the game engine. ▾

- 🔑 Read a page
- 🔑 Read a page
- 🔑 Read a page
- 🔑 Read a page
- 🕒 Wiring laser-meteor collision detection into the game engine.
- ✅ Done

Here's the fix for Task 1. The bug is in `handle_events`: SPACE only does

`self.started=True`, and nothing fires after launch. The fix needs one new file and one replaced file.

1. Create `game/laser.py`

```
import pygame

class Laser:
    SPEED = 10

    def __init__(self, x, y):
        self.rect = pygame.Rect(x - 2, y - 14, 4, 16)

    def update(self):
        self.rect.y -= self.SPEED

    def off_screen(self):
        return self.rect.bottom < 0
```

```

def hits(self, meteor):
    # closest point on the laser rect to the meteor's center
    cx = max(self.rect.left, min(meteor.x, self.rect.right))
    cy = max(self.rect.top, min(meteor.y, self.rect.bottom))
    return (meteor.x - cx) ** 2 + (meteor.y - cy) ** 2 <=
meteor.radius ** 2

def draw(self, screen):
    pygame.draw.rect(screen, (255, 80, 80), self.rect,
border_radius=2)

```

2. Replace all of `game/game_engine.py`

```

import pygame
import random
from game.ship import Ship
from game.meteor import Meteor
from game.laser import Laser

WIDTH, HEIGHT=700, 520
FPS=60
BG=(8, 5, 20)
SHOOT_COOLDOWN=12 # frames between shots

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen=pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Meteor Dodge")
        self.clock=pygame.time.Clock()
        self.font=pygame.font.SysFont("monospace", 26, bold=True)
        self.big_font=pygame.font.SysFont("monospace", 46, bold=True)
        self.stars=
[(random.randint(0, WIDTH), random.randint(0, HEIGHT), random.randint(1, 3))
for _ in range(80)]
        self.reset()

```

```

def reset(self):
    self.ship=Ship(WIDTH//2,HEIGHT-80)
    self.meteors=[]
    self.lasers=[]
    self.shoot_cooldown=0
    self.timer=0
    self.spawn_interval=60
    self.score=0
    self.game_over=False
    self.started=False

def fire_laser(self):
    if self.shoot_cooldown>0: return

self.lasers.append(Laser(self.ship.rect.centerx,self.ship.rect.top))
    self.shoot_cooldown=SHOOT_COOLDOWN

def handle_events(self):
    for event in pygame.event.get():
        if event.type==pygame.QUIT: return False
        if event.type==pygame.KEYDOWN:
            if event.key==pygame.K_SPACE:
                if self.game_over: self.reset()
                elif not self.started: self.started=True
                else: self.fire_laser()
    return True

def update(self):
    if self.game_over or not self.started: return
    keys=pygame.key.get_pressed()
    self.ship.move(keys,WIDTH,HEIGHT)
    if self.shoot_cooldown>0: self.shoot_cooldown-=1
    self.timer+=1
    if self.timer>=self.spawn_interval:

```

```

        self.meteors.append(Meteor(WIDTH))
        self.timer=0
        self.spawn_interval=max(20,self.spawn_interval-0.3)

    for l in self.lasers: l.update()
    self.lasers=[l for l in self.lasers if not l.off_screen()]

    for m in self.meteors: m.update()

    # laser vs meteor collisions
    for l in self.lasers[:]:
        for m in self.meteors:
            if l.hits(m):
                self.meteors.remove(m)
                self.lasers.remove(l)
                break

    for m in self.meteors:
        if m.collides(self.ship.rect):
            self.game_over=True
    self.meteors=[m for m in self.meteors if not
m.off_screen(HEIGHT)]
    self.score+=1

def draw(self):
    self.screen.fill(BG)
    for sx,sy,sr in self.stars:
        pygame.draw.circle(self.screen,(200,200,220),(sx,sy),sr)
    for m in self.meteors: m.draw(self.screen)
    for l in self.lasers: l.draw(self.screen)
    self.ship.draw(self.screen)
    sc=self.font.render(f"Time: {self.score//60}s",True,
(200,200,240))
    self.screen.blit(sc,(10,10))
    if not self.started:

```

```

        msg=self.font.render("Press SPACE to launch",True,
(180,180,240))
        self.screen.blit(msg,(WIDTH//2-
msg.get_width()//2,HEIGHT//2))
        if self.game_over:
            ov=pygame.Surface((WIDTH,HEIGHT),pygame.SRCALPHA)
            ov.fill((0,0,0,150))
            self.screen.blit(ov,(0,0))
            m=self.big_font.render("DESTROYED!",True,(220,80,60))
            s=self.font.render(f"Survived {self.score//60}s | SPACE
to Restart",True,(200,200,200))
            self.screen.blit(m,(WIDTH//2-m.get_width()//2,HEIGHT//2-
40))

            self.screen.blit(s,(WIDTH//2-
s.get_width()//2,HEIGHT//2+20))
            pygame.display.flip()

    def run(self):
        running=True
        while running:
            running=self.handle_events()
            self.update()
            self.draw()
            self.clock.tick(FPS)
        pygame.quit()

```

3. Test and commit

Run `python main.py`. SPACE launches, then each SPACE press fires a red laser that destroys meteors on contact. Then:

```

git add .
git commit -m "Task 1: fix laser firing input bug"

```

Remember you must record the "before" video first, before you paste any of this in.

Tell me when this works, and I'll send Task 2 (meteor splitting). It will need a small change to `meteor.py`, so it'll be another separate commit.

10/7/2026, 2:03:26 PM